

Simple Wikipedia Search Engine

Aniruddha Pandey
apandey10@stevens.edu
20037146

November 20, 2025

Instructor: Reza Peyrovian <rpeyrovi@stevens.edu>

Course: CS 600-A Advanced Algorithm Design and Implementation

1 Project Overview

This project is a search engine built in Rust, designed to index and search a local copy of the Simple English Wikipedia. It consists of multiple components, starting with a high-performance web crawler.

2 Project Structure

```
.
├── crawler/
│   ├── Cargo.toml
│   └── src/
│       └── main.rs
├── client/
│   ├── package.json
│   └── src/
│       └── routes/
│           └── +page.svelte
├── data/
│   ├── crawled.lst
│   ├── docs.bin
│   ├── inverted_index.bin
│   ├── lemmatization-en.txt
│   ├── seed.lst
│   ├── stopwords-en.txt
│   ├── trie.bin
│   └── whitelist.txt
├── fixtures/
│   ├── 01_basic_search_input.txt
│   ├── 01_basic_search_output.json
│   └── ...
├── indexer/
│   ├── Cargo.toml
│   └── src/
│       └── main.rs
├── normalization/
│   ├── Cargo.toml
│   └── src/
│       └── lib.rs
├── server/
│   ├── Cargo.toml
│   └── src/
│       └── main.rs
├── wikipedia-simple-html-dump/
│   └── ... (HTML files)
├── generate_fixtures.py
└── README.tex
```

3 Components

3.1 Crawler

The crawler is responsible for traversing the local Wikipedia HTML files to discover and list all available articles.

How it Works

1. **Initialization:** Reads a seed file (`data/seed.lst`) to find the starting point (e.g., `simple/index.html`).
2. **Traversal:** It uses a Breadth-First Search (BFS) algorithm to visit pages.
3. **Parsing:** For each visited page, it parses the HTML to find `<a>` tags (links).
4. **Filtering:** It strictly filters for **relative local file paths**, ignoring external links (`http/https`) and anchors (`#`).
5. **Output:** It generates a list of all discovered unique file paths.

Output

The crawler produces a file named `crawled.lst` in the `data/` directory. This file contains the relative paths of all visited documents, stripped of the `wikipedia-simple-html-dump/` prefix to ensure portability.

Example output in `data/crawled.lst`:

```
simple/index.html
simple/articles/a/r/t/Article_Name.html
...
```

3.2 Normalization

The `normalization` component is a text processing library responsible for preparing raw text for indexing.

Pipeline

1. **Casefolding:** Converts all text to lowercase to ensure case-insensitive matching.
2. **Whitelist Check:** Checks tokens against a whitelist (`data/whitelist.txt`) to preserve special terms (e.g., "C++", ".NET") that would otherwise be split by the tokenizer.
3. **Tokenization:** Splits text into individual tokens (words) based on punctuation and whitespace.
4. **Stopword Removal:** Filters out common English filler words (e.g., "the", "is", "at") to reduce index size and improve relevance. Uses `data/stopwords-en.txt`.
5. **Lemmatization:** Reduces words to their root form (e.g., "running" → "run") using a dictionary-based approach sourced from `data/lemmatization-en.txt` ([Source](#)).

3.3 Indexer

The indexer builds an efficient inverted index from the processed documents.

Architecture

Following the standard inverted index design:

1. **Inverted Index (Occurrence Lists):** A binary file (`data/inverted_index.bin`) storing the list of postings for each term. Each posting contains the Document ID and the list of positions where the term appears (for window scoring).
2. **Document Metadata:** A binary file (`data/docs.bin`) storing metadata for each document, including its path, title, and length (for BM25 scoring).
3. **Compressed Trie:** An in-memory compressed trie (implemented using `trie-rs`) that maps every term in the vocabulary to the index of its occurrence list in the binary file. The trie is serialized to `data/trie.bin`.

Process

1. **Initialization:** Loads the `crawled.lst` to map Document IDs (line numbers) to file paths.
2. **Processing:** Iterates through every document, parses the HTML to extract text and title, and uses the `Normalizer` library to tokenize and stem the content.
3. **Index Construction:** Builds a temporary in-memory map of `Term` $\rightarrow$ `DocID` $\rightarrow$ `[Positions]`.
4. **Optimization:** Sorts postings by Document ID.
5. **Serialization:** Saves the occurrence lists, document metadata, and the Trie to disk using `bincode`.

3.4 Search & Ranking

The search component provides the user interface for querying the index and ranking results.

Boolean Logic

The search engine enforces a strict Boolean AND logic for multi-term queries. Given a query $Q = \{q_1, q_2, \dots, q_n\}$, the result set R is defined as the intersection of the posting lists for each term:

$$R = \bigcap_{i=1}^n \text{Postings}(q_i)$$

This ensures that every returned document contains all the terms in the query. The intersection is computed efficiently by sorting the posting lists by Document ID and using a linear scan algorithm.

Ranking Algorithm

The search engine uses a sophisticated ranking function that combines two relevance signals:

1. **BM25 Score:** A probabilistic information retrieval model that ranks documents based on the query terms appearing in each document.

The BM25 score for a document D and query Q is calculated as:

$$\text{BM25}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})}$$

Where the Inverse Document Frequency (IDF) is:

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)$$

Parameters:

- $f(q_i, D)$: Term frequency of q_i in D .
- $|D|$: Length of document D .
- avgdl: Average document length in the corpus.
- N : Total number of documents.
- $n(q_i)$: Number of documents containing q_i .
- $k_1 = 1.2$, $b = 0.75$: Tunable parameters.

2. **Window Score**: Measures the proximity of query terms within the document.

$$\text{Window}(D, Q) = \frac{|Q|}{\min_window(Q, D)}$$

Where $\min_window(Q, D)$ is the size of the smallest span of text in D containing all terms in Q .

Final Score Formula:

$$\text{Score}(D, Q) = \alpha \cdot \text{Window}(D, Q) + \beta \cdot \text{BM25}(D, Q)$$

Where α and β are weights for the respective scores.

Features

- **Ranked Results**: Returns results sorted by relevance score.
- **Fast Retrieval**: Uses optimized data structures for sub-millisecond query times.

3.5 Server (REST API)

The server component exposes the search functionality via a RESTful API using **actix-web**.

Endpoints

- GET `/search?q=<query>`: Performs a search and returns JSON results.

Response Format

```
{
  "query": "wikipedia",
  "results": [
    { "id": 1, "path": "simple/index.html" },
    ...
  ],
  "total_results": 9145,
  "time_taken_ms": 2.34
}
```

3.6 Client (Web UI)

The client is a modern web interface built with **SvelteKit** that consumes the REST API.

Features

- **Clean UI:** Simple search bar and results display.
- **Real-time Feedback:** Shows search time and total results found.
- **Direct Links:** Links directly to the local file paths (requires browser permission or local setup).

4 Usage

4.1 Prerequisites

- Rust toolchain (Cargo, rustc)
- Local Wikipedia dump extracted to `wikipedia-simple-html-dump/` directory
- Lemmatization dictionary at `data/lemmatization-en.txt` ([Source](#))
- Stopwords list at `data/stopwords-en.txt`
- Whitelist at `data/whitelist.txt`

4.2 Configuration

The crawler uses a `seed.lst` file in the `data/` directory to determine the entry point.

```
simple/index.html
```

4.3 Running the Crawler

To execute the crawler from the project root. This will traverse the `wikipedia-simple-html-dump` directory and generate `data/crawled.lst`.

```
cargo run --manifest-path crawler/Cargo.toml
```

4.4 Running the Indexer

To build the index from the crawled data. This will process the files listed in `crawled.lst` and generate `data/inverted_index.bin` and `data/trie.bin`.

```
cargo run --manifest-path indexer/Cargo.toml
```

4.5 Running the Server

To start the REST API server on `http://127.0.0.1:8080`.

```
cargo run --manifest-path server/Cargo.toml
```

4.6 Running the Client

To start the web interface. Ensure you have Node.js installed.

Important: The client requires a symbolic link to the Wikipedia dump to serve the files locally.

1. Create the symbolic link (if not already created):

```
# Run from the project root
ln -s "../../wikipedia-simple-html-dump" "client/static/wiki"
```

2. Install dependencies and run the development server:

```
cd client
npm install
npm run dev
```

The client will be available at `http://localhost:5173`. Clicking on search results will open the local Wikipedia pages directly in your browser.

4.7 Running Tests

To run the unit tests for the Normalizer library (lemmatization, tokenizer, etc.):

```
cargo test --manifest-path normalization/Cargo.toml
```

4.8 Generating Test Fixtures

A Python script is included to generate a set of test cases for the search API, covering various boundary conditions. This script requires the server to be running.

1. Ensure the server is running (see above).
2. Run the script from the project root:

```
python3 generate_fixtures.py
```

This will create a `fixtures/` directory populated with input (`.txt`) and output (`.json`) files for each test case, such as basic searches, empty queries, and queries with special characters.

5 Data Source

The data used for this search engine is sourced from the Wikimedia Dumps, specifically the Simple English Wikipedia Database dump, which can be found at dumps.wikimedia.org. The lemmatization list is sourced from [michmech/lemmatization-lists](https://github.com/michmech/lemmatization-lists). The stopwords list is sourced from the `nltk` python library.

Academic Integrity Pledge

I pledge my honor that I have abided by the Stevens Honor System.^{[1](#)}

¹library.stevens.edu/academicintegrity